WPT Day 13 Morning

Mon 02 Nov 26 · Routing in React with React Router DOM (`BrowserRouter` , `Routes` , `Route`), `Link` vs the anchor tag, programmatic navigation with `useNavigate` , the automatic event object, and building a registration form with a single state object, a generic `handleChange` , the spread operator, dynamic keys and `preventDefault`

Table of Contents

1. What Is Routing and Why It Matters
2. Why React Needs a Third-Party Routing Library
3. Project Setup, Folder Structure and Page Components
4. Navbar: Anchor Tags vs the Link Component
5. Wiring Everything Together in App.jsx
6. Programmatic Navigation with useNavigate
7. The Automatic Event Object
8. Forms in React: The Three-Step Pattern
9. Combined Worked Example
10. Q&A and Next Steps
11. Quick Revision Checklist
12. Practice Exercises
13. Practice Answers

What Is Routing and Why It Matters

Problem: one app, many screens

Real-world apps have many screens: dashboard, login, profile, settings, contact us, registration, and so on. All of these **cannot be stacked on a single component**; each needs its own dedicated view. Without a mechanism to choose which view is visible, you would have either one giant component or no way to move between screens.

Mechanism

Routing is the mechanism that **opens the correct component based on the current browser URL**.

- Every screen maps to a specific **route** (a URL).
- Navigating to that URL renders the corresponding component.
- Gmail, Google Drive and YouTube all use routing to switch between views **without full page reloads**. Changing a tab or section changes the URL, and a different component renders for each.

Two ways a user can land on a route:

Way	Example
Typing the URL directly	Typing `/contact` in the address bar
Clicking a navigation item	Navbar, sidebar or button

> *Background:* In a Single Page Application (SPA) the browser downloads one HTML page and JavaScript swaps the visible components. The URL in the address bar is kept in sync with what is shown, so bookmarks, the back button and shared links keep working even though no new page is fetched from the server.

The division of labour:

```
User clicks navbar item
        |
        v
Navbar changes the URL          ⟵ navbar's ONLY job
        |
        v
React Router DOM looks at URL   ⟵ decides which component
        |
        v
Matching component is rendered
```

> **Exam:** Routing = opening a component on a specific URL. The navbar's sole job is to change the URL; deciding which component to render is React Router DOM's job.

Why React Needs a Third-Party Routing Library

Problem

React does not know anything about URLs. It renders components; it does not choose between screens.

Library vs framework

React is described as a **small, focused library, not a full framework**.

React provides	React does NOT provide
State	Routing
Props	HTTP clients
Lifecycle methods	Form validation
Hooks	Global state management

A **framework** (e.g., Angular) ships with routing built in; a **library** (React) expects you to compose your own stack. For routing, the third-party package is **React Router DOM**:

```
npm i react-router-dom
```

The three core components

Component	Role
`BrowserRouter`	Wraps the **entire app**; signals that browser-based routing is enabled
`Routes`	A **container** that holds all individual `Route` definitions
`Route`	Defines a **single URL-to-component mapping** via the `path` and `element` props

> **Exam:** The mandatory nesting is `BrowserRouter` → `Routes` → `Route`. React has no built-in routing; React Router DOM fills that gap.

Project Setup, Folder Structure and Page Components

A new Vite + React project was created for this session:

```
npx create-vite@latest routing-demo
cd routing-demo
npm i
npm run dev
```

Recommended folder structure inside `src/`

```
src/
├── components/     reusable UI pieces used across screens (e.g. Navbar)
```

```
│   └── Navbar.jsx
├── pages/          full screen-level components
│   ├── Home.jsx
│   ├── AboutUs.jsx
│   ├── ContactUs.jsx
│   ├── UserDetails.jsx
│   └── RegistrationForm.jsx
└── App.jsx         root component: Navbar + Routes
```

Keeping **pages** and **reusable components** separate keeps the project organised and scalable.

Page components

Three basic screens live in `src/pages/` (plus `UserDetails`, used later for the programmatic navigation demo).

`Home.jsx` (initial version):

```jsx
export function Home() {
  return (
    <div>
      <h3>Welcome to Home Page</h3>
      <p>Some random description about the home page.</p>
    </div>
  );
}
```

`AboutUs.jsx`:

```jsx
export function AboutUs() {
  return (
    <div>
      <h2>About Us</h2>
      <h4>Our Mission</h4>
      <p>Description of our mission.</p>
      <h4>Our Vision</h4>
      <p>Description of our vision.</p>
    </div>
  );
}
```

`ContactUs.jsx`:

```jsx
export function ContactUs() {
  return (
    <div>
      <h2>Contact Us</h2>
      <p>Phone: 9999999999</p>
      <p>Email: admin@gmail.com</p>
```

```
    </div>
  );
}
```

`UserDetails.jsx` :

```
export function UserDetails() {
  return (
    <div>
      <h2>User Details</h2>
      <p>See the user details below.</p>
      <table>
        <tr><th>ID</th><th>Name</th></tr>
        <tr><td>1</td><td>Suresh</td></tr>
        <tr><td>2</td><td>Ramesh</td></tr>
      </table>
    </div>
  );
}
```

Navbar: Anchor Tags vs the Link Component

Step 1 – Navbar with anchor tags (the initial, broken approach)

`src/components/Navbar.jsx` :

```
export function Navbar() {
  return (
    <div>
      <a href="/">Home</a>
      <a href="/about">About Us</a>
      <a href="/contact">Contact Us</a>
    </div>
  );
}
```

- Anchor tags **do change the URL correctly**, but they trigger a **full browser reload on every click**.
- Full reloads defeat the purpose of an SPA (all JavaScript restarts, all state is lost).
- The anchor tag's default behaviour is to **always request a new page from the server**.
- **The anchor tag is the culprit here, not React Router DOM.**

Step 2 – Navbar with `Link` (the correct approach)

React Router DOM provides a `Link` component as a **drop-in replacement for** `<a>`. It changes the URL **without reloading the browser**. Key difference: it uses a `to` prop instead of `href`.

```
import { Link } from 'react-router-dom';

export function Navbar() {
  return (
    <div>
      <Link to="/">Home</Link>
      <Link to="/about">About Us</Link>
      <Link to="/contact">Contact Us</Link>
    </div>
  );
}
```

After this change, navigating between pages updates the URL and renders the correct component with **zero browser reload**.

Aspect	`<a href>`	`<Link to>`
Prop for destination	`href`	`to`
Changes the URL	Yes	Yes
Full page reload	**Yes** (asks server for a new page)	**No**
SPA-friendly	No	Yes
Comes from	HTML	`react-router-dom`

> **Pitfall:** Using `<a href>` inside a React SPA makes routing "work" but reloads the whole app on each click. Do not blame React Router; replace `<a>` with `<Link>`.

Wiring Everything Together in App.jsx

`App.jsx` is the root component. It renders the Navbar once and delegates the rest to routing.

- **Navbar appears on every page** because it lives **outside** the `Routes` block.
- Structure: `BrowserRouter` wraps everything → `Navbar` at the top → `Routes` contains all `Route` definitions.

```jsx
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import { Navbar } from './components/Navbar';
import { Home } from './pages/Home';
import { AboutUs } from './pages/AboutUs';
import { ContactUs } from './pages/ContactUs';
import { UserDetails } from './pages/UserDetails';
import { RegistrationForm } from './pages/RegistrationForm';

function App() {
  return (
    <BrowserRouter>
      <Navbar />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<AboutUs />} />
        <Route path="/contact" element={<ContactUs />} />
        <Route path="/user-details" element={<UserDetails />} />
        <Route path="/register" element={<RegistrationForm />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

How it works

- `BrowserRouter` enables browser-history-based routing for the whole app.
- `Routes` acts as a container and **matches the current URL to exactly one** `Route`.
- Each `Route` takes a `path` (the URL) and an `element` (the component to render).
- The `path` in `Route` must **exactly match** the `to` value used in `Link`; a mismatch means the route is never matched.

Why not hardcode the component below Navbar?

Hardcoding `<Home />` below `<Navbar />` would **always show Home regardless of the URL**. The component shown must be **dynamic**, decided by the current route.

```
URL: /about
  |
BrowserRouter
  ├── Navbar            (always visible)
  ├── Routes
        ├── Route "/"            → Home        (no match)
        ├── Route "/about"       → AboutUs     (MATCH → rendered)
        ├── Route "/contact"     → ContactUs   (no match)
```

Programmatic Navigation with useNavigate

Problem

`Link` works when the user **clicks** a navbar item. But some scenarios require redirecting via **code**:

- After a **successful login** → automatically redirect to the dashboard.
- After completing a **multi-step form** → redirect to a confirmation page.
- Inside a `useEffect` → redirect based on a condition or API response.

Showing a "Click here to go to dashboard" link after login is bad UX; the redirect should be automatic.

Mechanism

- `useNavigate` is a **hook** from React Router DOM.
- Calling it **returns a** `navigate` **function**.
- Calling `navigate('/some-url')` redirects the user to that route.

`Home.jsx` updated:

```jsx
import { useNavigate } from 'react-router-dom';

export function Home() {
  const navigate = useNavigate();

  function viewUserDetailsHandler(event) {
    // React automatically passes the event object to any function called on an event
    console.log(event); // logs the SyntheticBaseEvent (click event object)

    // Any logic can run here before navigating
    navigate('/user-details');
  }

  return (
    <div>
      <h3>Welcome to Home Page</h3>
      <p>Some random description about the home page.</p>
      <button onClick={viewUserDetailsHandler}>View User Details</button>
    </div>
  );
}
```

- `useNavigate` **returns** the `navigate` function; it is not imported directly as a standalone function.
- `navigate('/user-details')` sends the user to that route **without a page reload**.

- The `Route` for `/user-details` must still be registered in `App.jsx` (done above).

	`Link`	`useNavigate`
Triggered by	User clicking a link	Code (after logic, API call, effect)
Form	Component in JSX	Hook returning a function
Typical use	Navbar, sidebar	Post-login redirect, post-form redirect

> **Exam:** `useNavigate` is a hook; it returns the `navigate` function. Use `navigate('/path')` for programmatic redirects.

The Automatic Event Object

Problem

In `viewUserDetailsHandler(event)` the function declares an `event` parameter, yet nobody passes one. Where does it come from?

Mechanism

Whenever a function is called **on an event** (`onClick` , `onChange` , `onSubmit` , `onKeyDown` , ...), React **automatically passes one argument** to it: the **event object**, representing the event that just occurred.

Event	Object received
`onClick`	click event object
`onChange`	change event object
`onSubmit`	submit event object

This happens because **React calls the function internally**; the developer only passes a **reference**, not a manual call.

```
// Passing a reference (correct):
<button onClick={viewUserDetailsHandler}>Click</button>
```

```
// Calling manually (wrong - executes immediately on render):
<button onClick={viewUserDetailsHandler()}>Click</button>
```

Because React calls the function, React also injects the event object as the **first argument** automatically.

> **Pitfall:** `onClick={handler()}` runs the handler during rendering (and passes the *return value* to onClick). Write `onClick={handler}`.

Forms in React: The Three-Step Pattern

```
Step 1: Create state to hold form data        (single object)
Step 2: Capture input live with onChange      (generic handleChange)
Step 3: Read state on submit, preventDefault  (handleSubmit)
```

Step 1 – Create state to hold form data

Form data must live in **component state**, not be read from the DOM at submit time. Two options:

Option	Description
A	One separate `useState` per field (name, phone, email, password)
B	**One single state object** containing all fields as key-value pairs

Option B is strongly preferred: fewer state variables = less complexity, fewer unnecessary re-renders, and the entire form data is in one place, easy to pass as a request body.

```
import { useState } from 'react';

const [formData, setFormData] = useState({
  name: '',
  phoneNumber: '',
  email: '',
  password: ''
});
```

Keys are initialised with **empty strings** because no user input exists yet.

Step 2 – Capture input as the user types (`onChange`)

- Data is **NOT captured on submit**; it is captured **live as the user types**.

- `onChange` fires every time the user adds or removes a character in an input.
- It is equivalent to `onkeyup` from core HTML/JavaScript.

Reading the field value: `event.target`

In core JavaScript you used `document.getElementById('fieldId').value`. In React the same result comes from the event object:

Expression	Meaning
`event.target`	The input element on which the event fired (equivalent to the `getElementById` result)
`event.target.value`	The current value typed into that field
`event.target.name`	The input's `name` attribute (used as the state key)

Why a single generic `handleChange`

The naive approach defines one handler per field:

```
// Naive approach - do NOT do this for many fields
function handleNameChange(event) { /* ... */ }
function handlePhoneChange(event) { /* ... */ }
function handleEmailChange(event) { /* ... */ }
function handlePasswordChange(event) { /* ... */ }
```

All four have **identical logic**; only the key differs. For 50 inputs that means 50 functions. Solution: **one generic** `handleChange` used by all inputs.

The spread operator problem (the wrong version first)

```
// WRONG - do not do this
function handleChange(event) {
  setFormData({ name: event.target.value });
}
```

`setFormData` **replaces the entire state object** with whatever is passed. This call wipes out `phoneNumber`, `email` and `password`, leaving only `name`. The fix: **spread the existing** `formData` **first, then override only the relevant key.**

```
formData = { name:'A', phoneNumber:'1', email:'e', password:'p' }

WRONG : setFormData({ name:'Ab' })
        → { name:'Ab' }                          (others LOST)
```

```
RIGHT : setFormData({ ...formData, name:'Ab' })
        → { name:'Ab', phoneNumber:'1', email:'e', password:'p' }
```

Dynamic key with square brackets

The key to update must be **dynamic**, not hardcoded. Give each input a `name` attribute that matches the corresponding state key:

Field	Attribute
name	`name="name"`
phone	`name="phoneNumber"`
email	`name="email"`
password	`name="password"`

Inside `handleChange`, read `event.target.name` to get the field's identifier, and use **square-bracket notation** `[event.target.name]` to use that value as a dynamic key.

The correct generic `handleChange`

```
function handleChange(event) {
  setFormData({
    ...formData,                              // preserve all existing key-value pairs
    [event.target.name]: event.target.value   // override only the field that changed
  });
}
```

- `...formData` spreads all existing keys into the new object.
- `[event.target.name]` dynamically sets the key (e.g., `"email"` when the email field changes).
- `event.target.value` is the new value the user just typed.

Step 3 – Read state on submit and prevent the default reload

- Clicking the submit button fires the form's `onSubmit` event.
- **Default browser behaviour on form submit: reload the page** (losing all state).
- Call `event.preventDefault()` **first** to stop the reload.

```
function handleSubmit(event) {
  event.preventDefault();   // stops the default browser reload
  console.log(formData);    // formData now contains the complete, populated form
object
```

```
  // Next step: use formData as the request body for a POST API call
}
```

Exam: `onChange` populates state live; `onSubmit` just reads it. `event.preventDefault()` stops the reload; the spread operator preserves other keys; `[event.target.name]` makes the key dynamic.

Combined Worked Example

This is one complete project tying the day together: a router, a Navbar using `Link`, a Home page with `useNavigate`, and the registration form. Create a Vite React project, run `npm i react-router-dom`, and use these files.

`src/components/Navbar.jsx`

```jsx
import { Link } from 'react-router-dom';

export function Navbar() {
  return (
    <div>
      <Link to="/">Home</Link> |{' '}
      <Link to="/about">About Us</Link> |{' '}
      <Link to="/contact">Contact Us</Link> |{' '}
      <Link to="/register">Register</Link>
    </div>
  );
}
```

`src/pages/Home.jsx`

```jsx
import { useNavigate } from 'react-router-dom';

export function Home() {
  const navigate = useNavigate();

  function viewUserDetailsHandler(event) {
    console.log(event);          // click event object, passed automatically by React
    navigate('/user-details');   // programmatic redirect
  }

  return (
    <div>
      <h3>Welcome to Home Page</h3>
```

```
      <p>Some random description about the home page.</p>
      <button onClick={viewUserDetailsHandler}>View User Details</button>
    </div>
  );
}
```

`src/pages/AboutUs.jsx`, `ContactUs.jsx`, `UserDetails.jsx` as shown earlier.

`src/pages/RegistrationForm.jsx`

```
import { useState } from 'react';

export function RegistrationForm() {
  // Step 1: one state object for the whole form
  const [formData, setFormData] = useState({
    name: '',
    phoneNumber: '',
    email: '',
    password: ''
  });

  // Step 2: one generic handler for every input
  function handleChange(event) {
    setFormData({
      ...formData,
      [event.target.name]: event.target.value
    });
  }

  // Step 3: read state on submit, stop the reload
  function handleSubmit(event) {
    event.preventDefault();
    console.log(formData); // ready to be sent as a POST request body
  }

  return (
    <div>
      <h3>Register</h3>
      <form onSubmit={handleSubmit}>
        <div>
          <label>Name</label>
          <input type="text" name="name" placeholder="Enter name" onChange=
{handleChange} />
        </div>
        <div>
          <label>Phone Number</label>
          <input type="text" name="phoneNumber" placeholder="Enter phone" onChange=
{handleChange} />
        </div>
        <div>
```

```
          <label>Email</label>
          <input type="email" name="email" placeholder="Enter email" onChange=
{handleChange} />
        </div>
        <div>
          <label>Password</label>
          <input type="password" name="password" placeholder="Enter password"
onChange={handleChange} />
        </div>
        <button type="submit">Sign Up</button>
      </form>
    </div>
  );
}
```

`src/App.jsx`

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import { Navbar } from './components/Navbar';
import { Home } from './pages/Home';
import { AboutUs } from './pages/AboutUs';
import { ContactUs } from './pages/ContactUs';
import { UserDetails } from './pages/UserDetails';
import { RegistrationForm } from './pages/RegistrationForm';

function App() {
  return (
    <BrowserRouter>
      <Navbar />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<AboutUs />} />
        <Route path="/contact" element={<ContactUs />} />
        <Route path="/user-details" element={<UserDetails />} />
        <Route path="/register" element={<RegistrationForm />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

End-to-end trace

1. The browser loads `/`. `BrowserRouter` reads the URL; `Navbar` renders; `Routes` matches `path="/"` and renders `Home`.
2. The user clicks **View User Details**. React calls `viewUserDetailsHandler(event)`, passing the click event. It logs the event and calls `navigate('/user-details')`.

3. The URL changes to `/user-details` **without reload**; `Routes` now matches that `Route` and renders `UserDetails`. The Navbar stays because it is outside `Routes`.
4. The user clicks **Register** (a `Link to="/register"`). The URL changes, `RegistrationForm` renders with `formData` all empty strings.
5. The user types `S` in Name. `onChange` fires; `handleChange` reads `event.target.name === "name"` and `value === "S"`; `setFormData({ ...formData, name: 'S' })` keeps the other three keys.
6. The user fills the other fields the same way; each keystroke re-renders with a fuller `formData`.
7. The user clicks **Sign Up**. `onSubmit` fires `handleSubmit`; `preventDefault()` stops the reload; the console shows the complete object, e.g. `{ name: 'Suresh', phoneNumber: '9999999999', email: 'a@b.com', password: '123' }`.

Q&A and Next Steps

- **API integration:** connecting the form data to a backend API (a POST request with `formData` as the request body) will be covered in the next session.
- **Payment integration:** not in scope for this course. Third-party gateways (e.g., PayPal, Razorpay) expose their own APIs; integration involves calling those APIs and handling webhook verification.
- **PDF / invoice generation in Node + Express:** not covered, but students can explore independently. Libraries like **PDFKit** build PDF documents programmatically: set content, font size, write data to the document, and send the **file (not JSON)** in the response. Search "Generate PDF in Node Express".
- **Student question:** how to limit repeated state updates while typing. A concept (likely **debouncing**) was raised, but the name could not be recalled; it was left unanswered and to be explored independently.

Quick Revision Checklist

Topic	One-line takeaway
Routing	Opens a component based on the current URL
Why a library	React has no built-in routing; install `react-router-dom`
Core components	`BrowserRouter` → `Routes` → `Route` (`path` + `element`)

Topic	One-line takeaway
Folder structure	`components/` reusable pieces, `pages/` full screens
Anchor tag	Changes URL but reloads the whole page; breaks SPA
`Link`	Drop-in `<a>` replacement; uses `to` ; no reload
Navbar placement	Outside `Routes`, so it shows on every page
Path matching	`Route path` must exactly match `Link to`
`useNavigate`	Hook returning `navigate('/url')` for code-driven redirects
Event object	React auto-passes it as first argument to event handlers
Reference vs call	`onClick={fn}` not `onClick={fn()}`
Form state	One state object, not one state per field
`onChange`	Fires on every keystroke (like `onkeyup`)
`event.target`	The element; `.value` reads it, `.name` identifies it
Spread operator	`...formData` preserves other keys
Dynamic key	`[event.target.name]` with matching `name` attributes
`preventDefault()`	Stops the submit reload

Practice Exercises

Concept

1. Define routing and state the navbar's job versus React Router DOM's job.
2. Why does React need a third-party package for routing? Name two other things React does not provide.
3. Name the three core React Router DOM components and what each does.
4. Why does a navbar built with `<a href>` defeat an SPA, and whose fault is it?
5. Why is Option B (single state object) preferred over one `useState` per field?

Application

6. Write the `Route` that renders `ContactUs` at `/contact`, and the `Link` that points to it.

7. Write a component with a button that redirects to `/dashboard` using `useNavigate`.
8. Write a generic `handleChange` for a form with fields `name` and `email`, and state which attribute each input must have.
9. List three situations where `useNavigate` is better than `Link`.

Predict the output

10. State is `{ name: 'A', email: 'a@x.com' }`. The user types in the name field and the handler is `setFormData({ name: event.target.value })`. What is the state afterwards?
11. A page has `<button onClick={go()}>Go</button>` where `go` calls `navigate('/about')`. What happens on first render?
12. The Navbar is placed *inside* a `Route` element for `/` only. On `/about`, is the Navbar visible?

Spot the bug

13. `<Link href="/about">About</Link>` — what is wrong?
14. `<Route path="/contact-us" element={<ContactUs />} />` with `<Link to="/contact">`. Why does nothing render?
15. A `handleSubmit` that only does `console.log(formData)` causes the page to reload and the state to vanish. Fix it.
16. `const navigate = useNavigate;` then `navigate('/home')` throws an error. Why?

Practice Answers

1. Routing is opening the correct component based on the current browser URL. The navbar only changes the URL; React Router DOM decides which component to render.

2. React is a small library, not a framework; it provides state, props, lifecycle and hooks only. It does not provide routing, so `react-router-dom` is installed. Other things it lacks: HTTP clients, form validation, global state management (any two).

3. `BrowserRouter` wraps the app and enables browser-based routing; `Routes` is a container of all `Route`s that matches the URL to one; `Route` maps a `path` to an `element`.

4. Anchors always request a new page from the server, causing a full reload that restarts the app and loses state. The anchor tag is the culprit, not React Router DOM; use `Link`.

5. Fewer state variables, less complexity, fewer unnecessary re-renders, and all form data is in one place, ready to be used as a request body.

6.

```
<Route path="/contact" element={<ContactUs />} />
<Link to="/contact">Contact Us</Link>
```

7.

```
import { useNavigate } from 'react-router-dom';

export function GoToDashboard() {
  const navigate = useNavigate();
  function handleClick() {
    navigate('/dashboard');
  }
  return <button onClick={handleClick}>Go to Dashboard</button>;
}
```

8.

```
function handleChange(event) {
  setFormData({
    ...formData,
    [event.target.name]: event.target.value
  });
}
```

The inputs need `name="name"` and `name="email"`, matching the state keys, plus `onChange={handleChange}`.

9. After a successful login (redirect to dashboard), after a multi-step form (confirmation page), inside a `useEffect` based on a condition or API response.

10. `{ name: 'Ab' }` (for example): `email` is lost because `setFormData` replaces the whole object and nothing was spread.

11. The handler runs immediately during render (it is called, not referenced), so the app redirects to `/about` right away instead of waiting for a click. Correct form: `onClick={go}`.

12. No. The Navbar must be outside `Routes` to appear on every page; inside a `Route` it renders only when that route matches.

13. `Link` uses the `to` prop, not `href`. Correct: `<Link to="/about">About</Link>`.

14. The `path` (`/contact-us`) does not exactly match the `to` value (`/contact`), so the route is never matched.

15. Add `event.preventDefault();` as the first line of `handleSubmit(event)`, which stops the default form-submit reload.

16. `useNavigate` must be **called**: `const navigate = useNavigate();`. It returns the `navigate` function; without the call, `navigate` is just the hook itself.